



Федеральное агентство по рыболовству
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Астраханский государственный технический университет»

Система менеджмента качества в области образования, воспитания, науки и инноваций
ООО «ДКС РУС» по международному стандарту ISO 9001:2015

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОММУНИКАЦИЙ
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЗАЩИТА ИНФОРМАЦИИ

ОТЧЕТ

о выполнении лабораторной работы № 1
Протокол Диффи-Хеллмана

Выполнил:
студент гр. ЗИНРБ-51

_____ Генералов В. В.

«_____» _____ 2026 г.

Преподаватель:

_____ Савоськин И. А.

«_____» _____ 2026 г.

Астрахань – 2026

ВВЕДЕНИЕ

Протокол Диффи-Хеллмана позволяет двум сторонам выработать общий секретный ключ при обмене по незащищённому каналу связи только открытыми значениями. Выработанный ключ затем используется симметричным шифром для защиты дальнейшего обмена.

Цель работы: «изучить и проанализировать работу протокола безопасного обмена ключами по незащищенному каналу связи Диффи-Хеллмана. Написать программу, реализующую работу протокола».

Задание: «Разработать программу, состоящую из частей Боба и Алисы (А и В). Один из участников генерирует ключ (< 10000) случайным образом, а затем организует передачу другому согласно протоколу Диффи-Хеллмана. В результате оба участника сеанса связи получают общий секретный ключ **К**. Числовые значения должны генерироваться случайным образом или вводиться с клавиатуры по желанию пользователя».

Для выполнения задания на языке Python 3 разработана программа **dh.py**, которая использует только стандартную библиотеку и имеет режимы случайной генерации параметров, ручного ввода, самопроверки по таблице 1.1 методических указаний и перебора ключа третьим лицом. Листинг программы приведён в приложении 1.

Работа программы подтверждена самопроверкой по всем десяти строкам таблицы 1.1 методических указаний (итог 10/10), пятью сеансами со случайными параметрами, проверкой реакции на ошибочный ручной ввод и восстановлением ключа перебором по журналу открытого канала. В отчёте также даны ответы на контрольные вопросы и рассмотрена основная уязвимость протокола – атака «человек посередине».

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Открытые параметры протокола – большое простое число p и порождающий элемент α мультипликативной группы Z_p^* : его порядок, то есть наименьшее натуральное m , при котором $\alpha^m \equiv 1 \pmod{p}$, равен $p - 1$. Порождающих элементов в группе Z_p^* ровно $\varphi(p - 1)$, где φ – функция Эйлера.

В методических указаниях участники обозначены буквами А и В; в отчёте они названы Алисой и Бобом, а буквами A и B , как и в программе, обозначены их открытые значения. Алиса и Боб выбирают случайные секретные числа x и y (далее – секретные показатели). Обмен показан на рисунке 1.1: Алиса посылает Бобу значение A , Боб Алисе – значение B :

$$A = \alpha^x \bmod p, \quad B = \alpha^y \bmod p, \quad (1.1)$$

где

A – открытое значение Алисы;

α – порождающий элемент группы Z_p^* ;

x – секретный показатель Алисы;

p – простой модуль;

B – открытое значение Боба;

y – секретный показатель Боба.

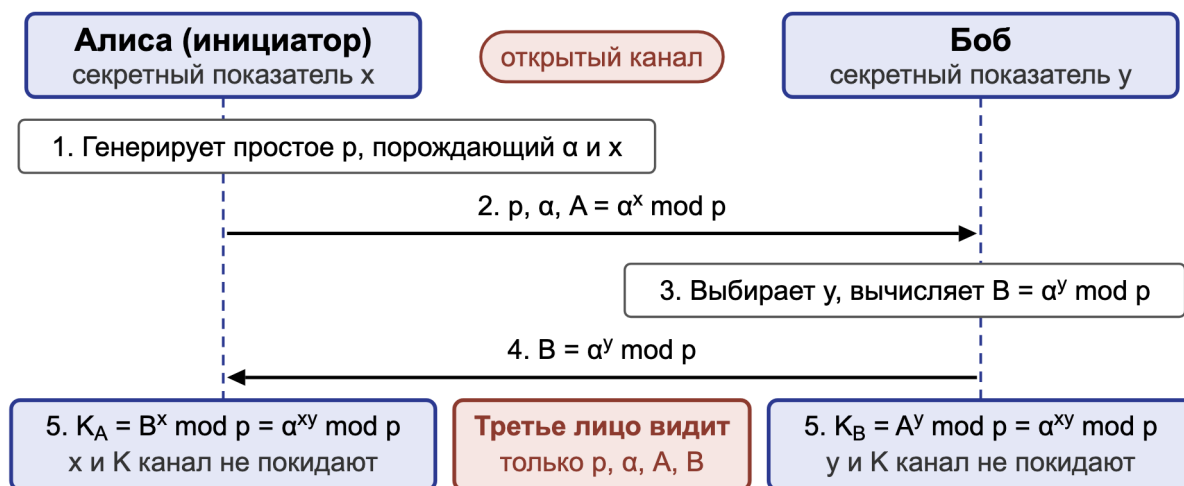


Рисунок 1.1 – Схема протокола Диффи-Хеллмана: что передаётся по открытому каналу

Алиса вычисляет ключ $K_A = B^x \bmod p$, Боб – ключ $K_B = A^y \bmod p$. Ключи совпадают:

$$K_A = (\alpha^y)^x \equiv \alpha^{xy} \equiv (\alpha^x)^y = K_B \pmod{p}. \quad (1.2)$$

По малой теореме Ферма $\alpha^{p-1} \equiv 1 \pmod{p}$, поэтому показатель степени можно приводить по модулю $p - 1$ и секретный показатель может превышать $p - 1$:

$$\alpha^x \equiv \alpha^{x \bmod (p-1)} \pmod{p}. \quad (1.3)$$

Для проверки работоспособности программы методические указания приводят тест самопроверки: исходные данные и ожидаемые значения ключа **K** приведены в таблице 1.1.

Таблица 1.1 – Тест самопроверки для протокола Диффи-Хеллмана

Строка	p	α	x	y	K
Строка 1	29	2	60	82	23
Строка 2	53	2	20	77	42
Строка 3	73	5	37	47	31
Строка 4	31	3	31	51	15
Строка 5	41	6	31	98	8
Строка 6	19	2	20	73	4
Строка 7	71	7	30	89	45
Строка 8	109	6	7	14	84
Строка 9	131	2	88	15	52
Строка 10	97	5	58	68	6

В строках 1, 2, 4–7 показатель x или y не меньше $p - 1$, что допустимо по формуле (1.3): в строке 1 $A = 2^{60} \bmod 29 = 2^4 \bmod 29 = 16$.

Стойкость протокола сводится к дискретному логарифмированию – нахождению x по p, α, A ; при $p < 10000$ оно выполняется перебором не более $p - 1$ степеней α за доли миллисекунды. Аутентификации сторон протокол не содержит: противник с секретным показателем z подменяет A и B значением $\alpha^z \bmod p$ и разделяет ключ с каждой стороной (атака «человек посередине»).

2 ОПИСАНИЕ АЛГОРИТМА И ПРОГРАММЫ

2.1 Постановка задачи

Из задания выделены семь требований: T1 – программа из частей Алисы и Боба с секретными показателями x и y ; T2 – случайная генерация при ограничении «< 10000»; T3 – по каналу проходят только p , α , $\alpha^x \bmod p$, $\alpha^y \bmod p$; T4 – выполняется $K_A = K_B$, о совпадении сообщается; T5 – ввод с клавиатуры; T6 – для десяти строк таблицы 1.1 ключ равен табличному K ; T7 – ответы на контрольные вопросы.

Там, где задание не уточняет условия, приняты допущения (константы P_MIN , P_MAX , функция *read_int*):

1. Ограничение «< 10000» отнесено к p из [5, 9999], откуда α , A , B , K меньше 10000.
2. Инициатор (Алиса) генерирует p , α , x и передаёт p , α , A , но не K .
3. Алиса и Боб – объекты одного процесса, канал – класс *Channel* с журналом.
4. Элемент α – порождающий (*is_generator*); при $p = 29$ таких элементов 12 из 28.
5. Показатели x , y сверху не ограничены по формуле (1.3), как требует таблица 1.1.
6. Ручной ввод проверяется (целое число; простое p из [5, 9999]; порождающий α из $[2, p - 2]$; $x, y \geq 1$), иначе запрос повторяется.

Требования подтверждены: T1 – рисунком 2.1, T2–T6 – в разделе 3, T7 – в разделе 4.

2.2 Описание алгоритма

Алиса генерирует p , α , x , Боб – y (см. рис. 1.1); по каналу передаются только p , α и значения A , B по формуле (1.1), а ключ K стороны вычисляют независимо по формуле (1.2).

Функция *gen_prime* выбирает случайное n из [5, 9999], пока оно не пройдёт проверку *is_prime* – пробное деление на нечётные d , не превышающие $\sqrt{n}$. Функция *element_order* находит порядок m элемента α прямым подсчётом за $O(p)$ шагов, *is_generator* требует $m = p - 1$. Порождающих элементов около трети (12/28, 32/96 и 3312/9972 для p , равного 29, 97 и 9973 соответственно), поэтому *gen_generator* тратит в среднем около трёх попыток.

Степени A , B , K вычисляет функция *modpow* бинарным методом с взятием остатка после каждого умножения (её код приведён на рисунке П1.1), поэтому промежуточные произведения не превышают $(p - 1)^2 < 2^{31}$. Вычислять α^x целиком нельзя: при p и x порядка 10^4 это число из десятков тысяч цифр, в C и Java возникает переполнение, а тип *double* теряет точность ($22^{60} \bmod 29$ даёт 12 вместо 23). Секретный показатель выбирается из $[1, p - 2]$ функцией *random.randint*, так как $x = 0$ и $x = p - 1$ дают $A = K = 1$; при ручном вводе достаточно $x \geq 1$ по формуле (1.3).

Функция *brute_force_dlog* моделирует третье лицо, прослушивающее канал: умножает на

α по модулю p до получения A и возвращает $k = x \bmod (p - 1)$. Этого достаточно, так как $K = B^k \bmod p$: для строки 1 таблицы 1.1 получены $k = 4 = 60 \bmod 28$ и $K = 23$.

2.3 Описание программы

Программа **dh.py** на языке Python 3 запускается командой «python3 dh.py», её состав показан на рисунке 2.1: арифметика – **modpow**, **is_prime**, **element_order**, **is_generator**, **gen_prime**, **gen_generator**; участники и канал – **Channel**, **Participant**, **Alice**, **Bob**, **run_protocol**, **print_session**; проверка и атака – **self_test**, **brute_force_dlog**, **attack_demo**; интерфейс – **read_int**, **manual_mode**, **random_mode**, **main**.

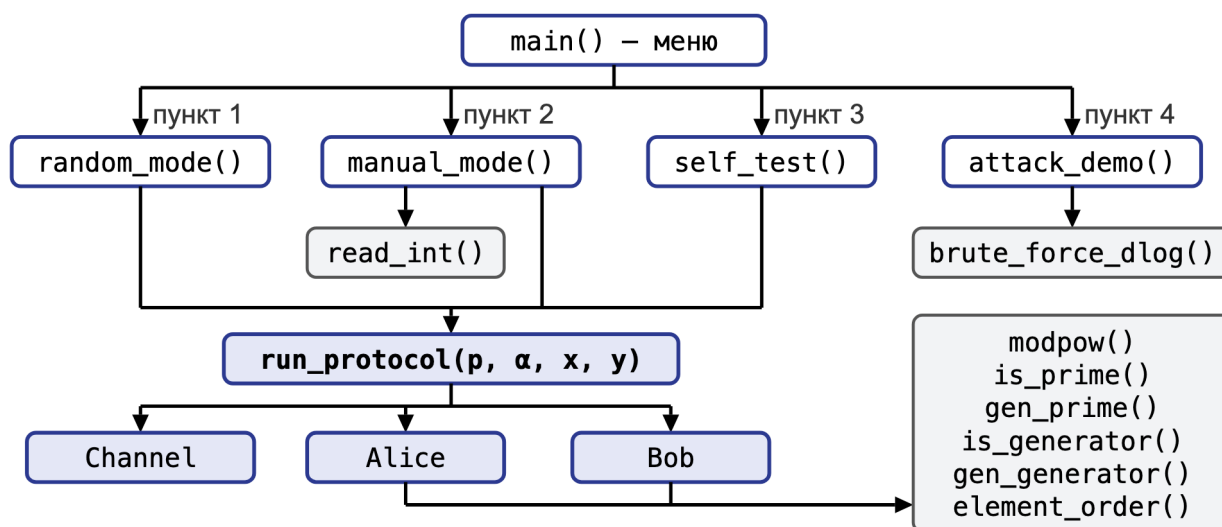


Рисунок 2.1 – Структура программы **dh.py**

Инициатор – Алиса: получает p и α , выбирает x , отправляет A ; Боб отвечает значением B . Через **Channel.send** проходят ровно четыре передачи – p , α , A , B ; x , y и ключи в журнал **log** не попадают.

Пункты меню программы: «1 — случайная генерация параметров» (**random_mode**), «2 — ручной ввод параметров» (**manual_mode**), «3 — самопроверка по таблице 1.1» (**self_test**), «4 — перебор ключа третьим лицом (по журналу последнего сеанса)» (**attack_demo**) и «0 — выход»; на другой ввод выводится «Нет такого пункта меню». Сеанс выводит журнал канала, секретные показатели, вычисления K_A , K_B и итог сравнения ключей.

В режиме «2» параметры читает функция **read_int**: нечисловой ввод даёт сообщение «Нужно ввести целое число.», нарушение условия – сообщение параметра: « p должно быть простым числом из [5, 9999].», « α должно лежать в [2, $p-2$] и быть порождающим элементом Z_p^* (порядок α равен $p-1$).», « x должно быть ≥ 1 .» (аналогично для y); затем запрос повторяется. Работа завершается пунктом «0», а «Ctrl + D» и «Ctrl + C» печатают «Выход.».

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Таблицы раздела получены прогоном функций программы из отдельного тестового сценария, рисунки – экраны программы.

3.1 Самопроверка по таблице методических указаний

Требование Т6 проверено функцией **self_test** (пункт меню «3»): каждая из десяти строк таблицы 1.1 выполнена функцией **run_protocol**, экран прогона приведён на рисунке 3.1.

Лабораторная работа №1. Протокол Диффи-Хеллмана

1 – случайная генерация параметров

2 – ручной ввод параметров

3 – самопроверка по таблице 1.1

4 – перебор ключа третьим лицом (по журналу последнего сеанса)

0 – выход

Выбор: 3

№	p	α	x	y	A	B	K ожид.	K получ.	результат
1	29	2	60	82	16	22	23	23	PASS
2	53	2	20	77	24	26	42	42	PASS
3	73	5	37	47	68	42	31	31	PASS
4	31	3	31	51	3	15	15	15	PASS
5	41	6	31	98	13	33	8	8	PASS
6	19	2	20	73	4	2	4	4	PASS
7	71	7	30	89	32	56	45	45	PASS
8	109	6	7	14	24	31	84	84	PASS
9	131	2	88	15	108	18	52	52	PASS
10	97	5	58	68	44	4	6	6	PASS

Итог: 10/10 строк совпали с таблицей 1.1

Рисунок 3.1 – Самопроверка по таблице 1.1: 10/10 PASS

Все строки получили отметку PASS: $K_A = K_B$ и равны ожидаемым K , итог 10/10. Поскольку ключи совпадают при любом α и даже при составном p , простота p и порядок α проверены отдельно. Все десять пар таблицы 1.1 проверку прошли (p простое, порядок α равен $p - 1$), контрпримеры приведены в таблице 3.1.

Таблица 3.1 – Контрпримеры к проверке простоты p и порождаемости α

p	p простое	α	Порядок α	$p - 1$	α порождающий
29	да	4	14	28	нет
7	да	2	3	6	нет
7	да	3	6	6	да
15	нет	2	–	14	–

При $p = 29$ элемент $\alpha = 4$ имеет порядок 14 вместо 28, при $p = 7$ порядки $\alpha = 2$ и $\alpha = 3$ равны 3 и 6, число $p = 15$ составное. Приведение показателя по модулю $p - 1$ ключа не меняет: для строки 1 таблицы 1.1 получены $60 \bmod 28 = 4$, $82 \bmod 28 = 26$ и $K = 23$.

3.2 Случайная генерация параметров

Требования Т2 и Т4 проверены пятью сеансами *run_protocol* без аргументов (пункт меню «1»), результаты приведены в таблице 3.2.

Таблица 3.2 – Пять сеансов в случайном режиме

p	α	x	y	A	B	K_A	K_B	$K_A = K_B$	p простое, α порождающий
1327	732	771	1052	59	1115	241	241	да	да
7499	5768	5869	4731	1372	4272	2006	2006	да	да
7559	3460	743	3267	2333	7126	1293	1293	да	да
9601	9086	8167	1908	1972	4784	8830	8830	да	да
7489	3847	5854	4470	6222	2033	3864	3864	да	да

Во всех сеансах $K_A = K_B$; модули p различны, простые и меньше 10000, элементы α – порождающие.

3.3 Ручной ввод параметров

Требование Т5 проверено функцией *manual_mode* в режиме ручного ввода (пункт меню «2»). Ввод строки 1 таблицы 1.1 и последующий перебор ключа по журналу канала (пункт меню «4») показаны на рисунке 3.2.

Выбор: 2 Ручной ввод параметров. p (простое, $5 \leq p \leq 9999$): 29 α (порождающий элемент, $2 \leq \alpha \leq p-2$): 2 x (секрет Алисы, $x \geq 1$): 60 y (секрет Боба, $y \geq 1$): 82 Обмен по открытому каналу: Alice -> Bob: $p = 29$ Alice -> Bob: $\alpha = 2$ Alice -> Bob: $A = \alpha^x \bmod p = 16$ Bob -> Alice: $B = \alpha^y \bmod p = 22$ секрет Алисы $x = 60$, секрет Боба $y = 82$ (по каналу не передаются) Алиса: $K = B^x \bmod p = 22^{60} \bmod 29 = 23$ Боб: $K = A^y \bmod p = 16^{82} \bmod 29 = 23$ Ключи совпали: общий сеансовый ключ $K = 23$ Выбор: 4 Перехвачено из канала: $p = 29$, $\alpha = 2$, $A = 16$, $B = 22$ Перебор: $x = 4$ (по модулю $p-1$) найден за 4 итераций, 0.001 мс Восстановленный ключ $K = B^x \bmod p = 23$

Рисунок 3.2 – Ручной ввод строки 1 таблицы 1.1 и перебор ключа

По каналу прошли только p, α, A, B (требование Т3). По перехваченным $p, \alpha, A = 16, B = 22$ найдено $k = 4$ ($k = x \bmod 28$) за четыре итерации и восстановлен ключ $K = 23$.

В таблице 3.3 приведена реакция функции **manual_mode** на шесть вариантов ввода, включая ошибочные.

Таблица 3.3 – Реакция программы на ручной ввод

Ввод	Ожидаемая реакция	Фактическая реакция
$p = 15$ (составное)	Отказ, повторный запрос p	Сообщение « p должно быть простым числом...», повторный запрос
$p = 29, \alpha = 4$ (порядок 14, не порождающий)	Отказ, повторный запрос α	Сообщение « α должно лежать в $[2, p-2]$ и быть порождающим...», повторный запрос
$p = 29, \alpha = 1$ (вне диапазона $[2, p-2]$)	Отказ, повторный запрос α	Сообщение « α должно лежать в $[2, p-2]$ и быть порождающим...», повторный запрос
$x = 0$	Отказ, повторный запрос x	Сообщение « x должно быть ≥ 1 ...», повторный запрос
$x = \text{«abc»}$ (нечисловой ввод)	Сообщение, повторный запрос x	Сообщение «Нужно ввести целое число...», повторный запрос
$p = 29, \alpha = 2, x = 60, y = 82$ ($x > p - 1$)	Принято, $K = 23$	Принято, $K = 23$

Составное $p = 15$, значения $\alpha = 4, \alpha = 1, x = 0$ и строка «abc» отвергнуты с повторным запросом; значения $x = 60, y = 82$ при $p = 29$ приняты, хотя больше $p - 1$, получен ключ $K = 23$.

3.4 Перебор ключа третьим лицом

Для ответа на контрольный вопрос 2 функция **brute_force_dlog** ищет наименьшее k , при котором $\alpha^k \equiv A \pmod{p}$, затем вычисляется $K = B^k \pmod{p}$. Результаты для трёх значений p (29, 997 и 9973) приведены в таблице 3.4.

Таблица 3.4 – Восстановление ключа третьим лицом перебором дискретного логарифма

p	α	x	A	Найденный k	Время, мс	K	Восстановленный K	Совпадение
29	2	60	16	4	0,005	23	23	да
997	123	328	88	328	0,014	778	778	да
9973	8387	1682	4610	1682	0,065	2043	2043	да

Ключ восстановлен за 4, 328 и 1682 итерации, время перебора – от 0,005 до 0,065 мс. Таблица 3.4 получена отдельным прогоном тестового сценария, а не при запуске, показанном на

рисунке 3.2, поэтому время для $p = 29$ в ней (0,005 мс) отличается от экранного (0,001 мс).

3.5 Анализ результатов

Все десять строк таблицы 1.1 воспроизведены (см. рис. 3.1), пять случайных сеансов (таблица 3.2) дали $K_A = K_B$.

Равенство ключей не доказывает корректность параметров: тождество (1.2) верно при любом α и составном p , поэтому в таблице 3.1 простота p и порождаемость α проверены отдельно. Порождающих элементов в Z_p^* от 33 до 43 %: 12/28 ($p = 29$), 32/96 ($p = 97$) и 3312/9972 ($p = 9973$), поэтому случайное α требует проверки. При $p = 29$ элемент $\alpha = 4$ имеет порядок 14: ключи совпадают, но перебор сокращается до 14 итераций; такое α отвергается (таблица 3.3).

При $p < 10000$ ключ восстанавливается перебором за доли миллисекунды (таблица 3.4: 0,065 мс при $p = 9973$). Перебор находит не сам x , а $k = x \bmod (p - 1)$, чего по малой теореме Ферма достаточно: в строке 1 таблицы 1.1 найдено $k = 4$ и восстановлен ключ $K = 23$ (см. рис. 3.2). Ограничение « < 10000 » – учебное; реальный протокол требует p длиной от 2048 бит.

Главная уязвимость – отсутствие аутентификации: активный противник подменяет A и B на $\alpha^z \bmod p$, получая ключ A^z с Алисой и B^z с Бобом, а стороны считают обмен успешным. Для защиты открытые значения аутентифицируют: подписями в протоколе Station-to-Station (STS), сертификатами в протоколе Transport Layer Security (TLS), отпечатками ключей в протоколе Secure Shell (SSH).

Таким образом, требование T1 подтверждено структурой программы (см. рис. 2.1), требования T2–T6 – таблицами 3.1–3.4 и экранами программы (см. рис. 3.1, 3.2), требование T7 – разделом 4.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Для чего используется протокол Диффи-Хеллмана? Для чего нужен общий сеансовый ключ K ? Протокол позволяет двум сторонам выработать общий секретный ключ по незащищённому каналу без заранее согласованного ключа: передаются только открытые p , α , A , B , а $K = \alpha^{xy} \bmod p$ стороны вычисляют сами. Ключ K используется быстрым симметричным шифром для защиты последующего обмена; «сеансовый» означает действующий в одном сеансе: новые x , y дают новый ключ (таблица 3.2).

Вопрос 2. Может ли третье лицо, прослушивая канал связи, получить общий сеансовый ключ K ? По каналу проходят только p , α , A , B ; получение K сводится к дискретному логарифмированию, при p длиной 2048 бит практически неосуществимому, поэтому ответ – нет. При учебных параметрах ответ обратный: функция *brute_force_dlog* перебирает k , пока не выполнится $\alpha^k \equiv A \pmod{p}$; для $p = 29$ показатель $x \bmod 28 = 4$ найден за четыре итерации, ключ $K = 23$ (см. рис. 3.2); по таблице 3.4 перебор занял 0,005 мс, при $p < 10000$ ключ вскрывается за доли миллисекунды. Стойкость требует порождающего α , случайных x , y и модуля $p = 2q + 1$ (с простым q) длиной не менее 2048 бит.

Вопрос 3. В чем заключается основная уязвимость протокола Диффи-Хеллмана? Чем он, как правило, дополняется? Основная уязвимость – отсутствие аутентификации сторон, откуда атака «человек посередине», показанная на рисунке 4.1: противник (Мэллори) подменяет A и B на $\alpha^z \bmod p$, получает ключ K_1 с Алисой и ключ K_2 с Бобом, где $K_1 = \alpha^{xz} \bmod p$, $K_2 = \alpha^{yz} \bmod p$, и перешифровывает сообщения; у сторон протокол завершается без ошибок.

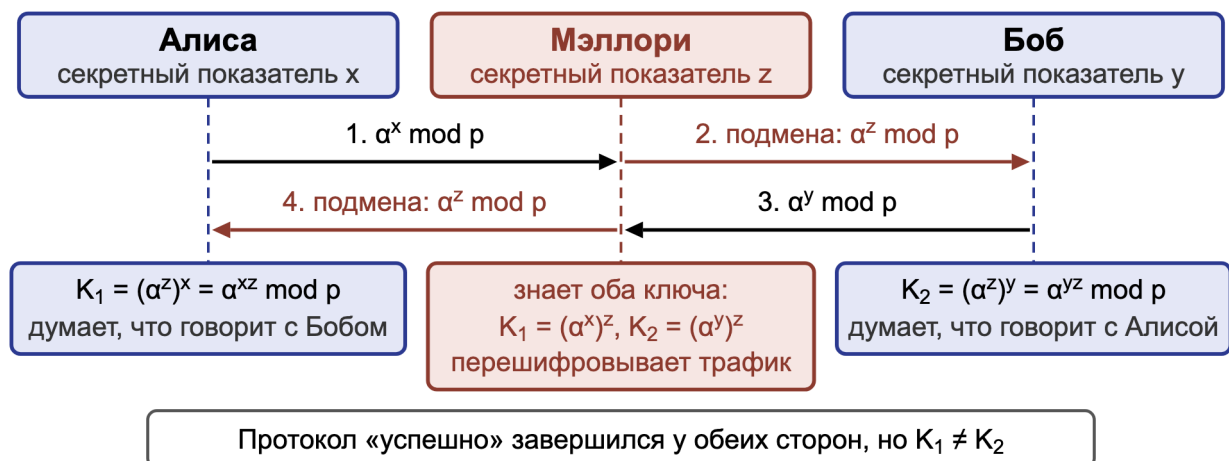


Рисунок 4.1 – Атака «человек посередине» на протокол без аутентификации

Поэтому протокол дополняется аутентификацией передаваемых значений: подписями открытых значений (STS), сертификатами (TLS), сравнением отпечатков ключей (SSH), а также протоколами обмена ключами на основе пароля (Password Authenticated Key Exchange, PAKE).

ЗАКЛЮЧЕНИЕ

В работе разработана программа **dh.py** со сторонами Алисы и Боба; канал передаёт только **p**, **α**, **A**, **B** (см. рис. 3.2). Самопроверка пройдена для 10/10 строк таблицы 1.1 (см. рис. 3.1); пять случайных сеансов дали простые **p** < 10000, порождающие **α** и равные ключи (таблица 3.2); ошибочный ввод отклоняется (таблица 3.3); ответы на контрольные вопросы приведены в разделе 4. Требования Т1–Т7 выполнены, цель работы достигнута.

Показатели **x**, **y**, не меньшие **p** – 1, допустимы по малой теореме Ферма; равенство **K_A** = **K_B** не гарантирует порождаемости **α** (таблица 3.1); при **p** < 10000 секретный показатель восстанавливается перебором (таблица 3.4), а без аутентификации возможна атака «человек посередине» (см. рис. 4.1).

Ограничения программы связаны с учебными параметрами: модуль меньше 10000 не даёт стойкости, реальный протокол требует **p** длиной от 2048 бит и аутентификации открытых значений.

ПРИЛОЖЕНИЕ 1

ЛИСТИНГ ПРОГРАММЫ

Листинг программы **dh.py** приведён на рисунке П1.1.

```
#!/usr/bin/env python3
"""Лабораторная работа №1. Протокол Диффи-Хеллмана.

Программа состоит из частей Алисы и Боба. Алиса (инициатор) генерирует
открытые параметры p и α и свой секретный показатель x, передаёт p, α и
 $A = \alpha^x \bmod p$  Бобу по открытому каналу. Боб выбирает секретный y, отвечает
 $B = \alpha^y \bmod p$ . Обе стороны вычисляют общий ключ K:
 $K_A = B^x \bmod p$ ,  $K_B = A^y \bmod p$ ,  $K_A == K_B == \alpha^{(xy)} \bmod p$ .

Режимы: случайная генерация параметров, ручной ввод с клавиатуры,
самопроверка по таблице 1.1 методички, перебор ключа третьим лицом
по журналу канала (почему учебное  $p < 10000$  не даёт стойкости).

Только стандартная библиотека. Запуск: python3 dh.py
"""

import random
import time

# --- Ограничения задания -----
# «Ключ (< 10000)» трактуем как ограничение на модуль p; тогда все значения,
# идущие по каналу (p, α, A, B), и сам ключ K тоже меньше 10000.
P_MIN = 5          # при p = 2, 3 нет ни одного α в диапазоне [2, p-2]
P_MAX = 9999

# Таблица 1.1 методички – тест самопроверки: (p, α, x, y, K)
TABLE_1_1 = [
    (29, 2, 60, 82, 23),
    (53, 2, 20, 77, 42),
    (73, 5, 37, 47, 31),
    (31, 3, 31, 51, 15),
    (41, 6, 31, 98, 8),
    (19, 2, 20, 73, 4),
    (71, 7, 30, 89, 45),
    (109, 6, 7, 14, 84),
    (131, 2, 88, 15, 52),
    (97, 5, 58, 68, 6),
]

# --- Арифметика по модулю -----
def modpow(base, exp, mod):
    """Быстрое возведение в степень по модулю (бинарный метод).

    Остаток берётся после каждого умножения, поэтому промежуточные значения
    не превышают  $(\text{mod}-1)^2$ . Считать  $\text{base}^{**\text{exp}}$  целиком не стоит: при  $p \sim 10^4$ 
    и  $x \sim 10^4$  это число из десятков тысяч цифр (в Java/C – переполнение,
    через double – потеря точности). Эквивалентно встроенному pow(base, exp, mod).
    """
    if mod == 1:
        return 0
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = result * base % mod
            base = base * base % mod
            exp >>= 1
    return result

def is_prime(n):
    """Проверка простоты пробным делением до sqrt(n). Для n < 10000 хватает."""
```

```

    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d += 2
    return True

def element_order(a, p):
    """Порядок элемента a в группе  $Z_p^*$ : наименьшее  $k \geq 1$  с  $a^k \equiv 1 \pmod{p}$ .

    Лобовой подсчёт: умножаем на a, пока не получим 1.  $O(p)$  шагов,
    для  $p < 10000$  это доли миллисекунды. Определено только для простого p:
    при составном p множество  $\{1..p-1\}$  не является группой по умножению.
    """
    if not is_prime(p):
        raise ValueError("порядок элемента определён только для простого p")
    a %= p
    if a == 0:
        raise ValueError("a должно быть ненулевым по модулю p")
    k, value = 1, a
    while value != 1:
        value = value * a % p
        k += 1
    return k

def is_generator(a, p):
    """a – порождающий элемент  $Z_p^*$ , если его порядок равен p-1,
    т.е. степени  $a^1..a^{p-1}$  пробегают все числа  $1..p-1$ ."""
    return 2 <= a <= p - 2 and element_order(a, p) == p - 1

def gen_prime(lo=P_MIN, hi=P_MAX):
    """Случайное простое из [lo, hi]."""
    while True:
        n = random.randint(lo, hi)
        if is_prime(n):
            return n

def gen_generator(p):
    """Случайный порождающий элемент  $Z_p^*$ . Генераторов ровно  $\phi(p-1)$  штук
    (в среднем около трети от p-1), поэтому нужно несколько попыток."""
    while True:
        a = random.randint(2, p - 2)
        if is_generator(a, p):
            return a

# --- Участники протокола и открытый канал -----
class Channel:
    """Открытый (прослушиваемый) канал: всё, что через него проходит,
    печатается и записывается в журнал – это ровно то, что видит третье лицо."""

    def __init__(self, verbose=True):
        self.verbose = verbose
        self.log = {} # метка -> значение, в порядке передачи

    def send(self, sender, receiver, label, value):
        self.log[label] = value
        if self.verbose:
            print(f" {sender} -> {receiver}: {label} = {value}")
        return value

```

```

class Participant:
    """Общая часть Алисы и Боба: секретный показатель и две операции протокола."""

    def __init__(self, name):
        self.name = name
        self.secret = None
        self.key = None
        # x у Алисы, у у Боба; канал не покидает
        # общий ключ K после завершения протокола

    def choose_secret(self, p, value=None):
        self.secret = value if value is not None else random.randint(1, p - 2)
        return self.secret

    def public_value(self, alpha, p):
        """ $\alpha^{\text{secret}} \bmod p$  – единственное, что участник отправляет в канал."""
        return modpow(alpha, self.secret, p)

    def compute_key(self, other_public, p):
        """ $K = (\alpha^{\text{other}})^{\text{secret}} \bmod p$ ."""
        self.key = modpow(other_public, self.secret, p)
        return self.key

class Alice(Participant):
    """Инициатор: генерирует p,  $\alpha$  (если не заданы) и начинает обмен."""

    def __init__(self):
        super().__init__("Alice")

    def setup(self, p=None, alpha=None):
        p = p if p is not None else gen_prime()
        alpha = alpha if alpha is not None else gen_generator(p)
        return p, alpha

class Bob(Participant):
    def __init__(self):
        super().__init__("Bob")

def run_protocol(p=None, alpha=None, x=None, y=None, verbose=True):
    """Полный сеанс протокола. Возвращает (p,  $\alpha$ , x, y, A, B, K_A, K_B, канал)."""
    alice, bob = Alice(), Bob()
    channel = Channel(verbose)

    p, alpha = alice.setup(p, alpha)
    channel.send("Alice", "Bob", "p", p)
    channel.send("Alice", "Bob", "alpha", alpha)

    x = alice.choose_secret(p, x)
    A = channel.send("Alice", "Bob", "A =  $\alpha^x \bmod p$ ", alice.public_value(alpha, p))

    y = bob.choose_secret(p, y)
    B = channel.send("Bob", "Alice", "B =  $\alpha^y \bmod p$ ", bob.public_value(alpha, p))

    k_a = alice.compute_key(B, p)
    k_b = bob.compute_key(A, p)
    # Алиса:  $K = B^x \bmod p$ 
    # Боб:  $K = A^y \bmod p$ 
    return p, alpha, x, y, A, B, k_a, k_b, channel

def print_session(p, alpha, x, y, A, B, k_a, k_b):
    print(f" секрет Алисы x = {x}, секрет Боба y = {y} (по каналу не передаются)")
    print(f" Алиса:  $K = B^x \bmod p = \{B\}^{\{x\}} \bmod \{p\} = \{k_a\}$ ")
    print(f" Боб:  $K = A^y \bmod p = \{A\}^{\{y\}} \bmod \{p\} = \{k_b\}$ ")
    if k_a == k_b:
        print(f" Ключи совпали: общий сеансовый ключ K = {k_a}")
    else:
        print(" ОШИБКА: ключи различны")
    if A == 1 or B == 1:
        print(" Внимание: открытое значение равно 1 – секретный показатель кратен p-1,")
        print(" ")

```

```

        "ключ вырожден (K = 1).")

# --- Самопроверка по таблице 1.1 -----
def self_test(verbose=True):
    """Прогон всех 10 строк таблицы 1.1. Возвращает число пройденных строк."""
    passed = 0
    if verbose:
        print(f" {'№':>2} {'p':>4} {'α':>2} {'x':>3} {'y':>3} {'A':>4} {'B':>4} {'K'
              'ожд.':>8} {'K получ.':>9} результат")
    for i, (p, alpha, x, y, k_expected) in enumerate(TABLE_1_1, 1):
        _, _, _, _, A, B, k_a, k_b, _ = run_protocol(p, alpha, x, y, verbose=False)
        ok = k_a == k_b == k_expected
        passed += ok
        if verbose:
            print(f" {i:>2} {p:>4} {alpha:>2} {x:>3} {y:>3} {A:>4} {B:>4}
                  {k_expected:>8} {k_a:>9} {'PASS' if ok else 'FAIL'}")
    if verbose:
        print(f" Итог: {passed}/{len(TABLE_1_1)} строк совпали с таблицей 1.1")
    return passed

# --- Перебор дискретного логарифма (к контрольному вопросу 2) -----
def brute_force_dlog(p, alpha, target):
    """По перехваченным p, α, A = α^x mod p найти x перебором k = 1, 2, ...

    Возвращает наименьший k с α^k ≡ A (mod p): это x mod (p-1), а не сам x,
    но ключ по нему восстанавливается верно. Число итераций равно найденному k.
    Работает только потому, что p мало."""
    value = 1
    for k in range(1, p):
        value = value * alpha % p
        if value == target:
            return k
    return None

def attack_demo(channel):
    """Третье лицо знает только журнал канала: p, α, A, B — и ничего больше."""
    p, alpha = channel.log["p"], channel.log["alpha"]
    A, B = channel.log["A = alpha^x mod p"], channel.log["B = alpha^y mod p"]
    t0 = time.perf_counter()
    x_found = brute_force_dlog(p, alpha, A)
    elapsed_ms = (time.perf_counter() - t0) * 1000
    k_found = modpow(B, x_found, p) if x_found is not None else None
    print(f" Перехвачено из канала: p = {p}, alpha = {alpha}, A = {A}, B = {B}")
    print(f" Перебор: x = {x_found} (по модулю p-1) найден за {x_found} итераций,
          {elapsed_ms:.3f} мс")
    print(f" Восстановленный ключ K = B^x mod p = {k_found}")
    return x_found, elapsed_ms, k_found

# --- Ввод с клавиатуры -----
def read_int(prompt, check=None, error="Недопустимое значение, повторите ввод."):
    """Читает целое число; при нечисловом вводе или нарушении check
    печатает сообщение и спрашивает снова."""
    while True:
        raw = input(prompt).strip()
        try:
            value = int(raw)
        except ValueError:
            print(" Нужно ввести целое число.")
            continue
        if check is None or check(value):
            return value
        print(" ", error)

def manual_mode():
    """Ручной ввод p, α, x, y с проверками. Возвращает канал сеанса."""

```

```

print("Ручной ввод параметров.")
p = read_int(f" p (простое, {P_MIN} <= p <= {P_MAX}): ",
             lambda v: P_MIN <= v <= P_MAX and is_prime(v),
             f"p должно быть простым числом из [{P_MIN}, {P_MAX}].")
alpha = read_int(" alpha (порождающий элемент, 2 <= alpha <= p-2): ",
                 lambda v: is_generator(v, p),
                 "alpha должно лежать в [2, p-2] и быть порождающим элементом Z_p*"
                 "
                 "(порядок alpha равен p-1).")
# Верхняя граница для x, y не проверяется:  $\alpha^x \equiv \alpha^{(x \bmod (p-1))} \pmod{p}$ ,
# в таблице 1.1 шесть строк из десяти имеют x или y >= p-1.
x = read_int(" x (секрет Алисы, x >= 1): ", lambda v: v >= 1, "x должно быть >=
1.")
y = read_int(" y (секрет Боба, y >= 1): ", lambda v: v >= 1, "y должно быть >=
1.")
print("Обмен по открытому каналу:")
p, alpha, x, y, A, B, k_a, k_b, channel = run_protocol(p, alpha, x, y)
print_session(p, alpha, x, y, A, B, k_a, k_b)
return channel

def random_mode():
    """Случайная генерация всех параметров. Возвращает канал сеанса."""
    print("Случайная генерация параметров (Алиса – инициатор).")
    print("Обмен по открытому каналу:")
    p, alpha, x, y, A, B, k_a, k_b, channel = run_protocol()
    yes_no = lambda flag: "да" if flag else "нет"
    print(f" свойства параметров: p простое – {yes_no(is_prime(p))}, p < 10000 –
    {yes_no(p < 10000)}, "
          f"alpha порождающий – {yes_no(is_generator(alpha, p))}")
    print_session(p, alpha, x, y, A, B, k_a, k_b)
    return channel

def main():
    last = None # канал последнего сеанса – для атаки
    menu = ("Лаб. работа №1. Протокол Диффи-Хеллмана\n"
            " 1 – случайная генерация параметров\n"
            " 2 – ручной ввод параметров\n"
            " 3 – самопроверка по таблице 1.1\n"
            " 4 – перебор ключа третьим лицом (по журналу последнего сеанса)\n"
            " 0 – выход")
    try:
        while True:
            print(menu)
            choice = input("Выбор: ").strip()
            if choice == "1":
                last = random_mode()
            elif choice == "2":
                last = manual_mode()
            elif choice == "3":
                self_test()
            elif choice == "4":
                if last is None:
                    last = random_mode()
                attack_demo(last)
            elif choice == "0":
                break
            else:
                print(" Нет такого пункта меню.")
    except (EOFError, KeyboardInterrupt):
        print("\nВыход.")

if __name__ == "__main__":
    main()

```

Рисунок П1.1 – Листинг программы *dh.py*